Technical Documentation

Fresh Cart

Project : Trinity - Part Three
(Mobile Grocery Platform)

Provided by:
Xinxin Miao
Boyu FU
Elie Zaylaa

Table of Contents

1. Executive Summary……3
2. Project Background & Objectives…….3
3. Requirements of Compliance Matrix……3
4. Overall Architecture……..5
5. Technology Stack & Architectural Decisions (ADR) ……6
6. Component & Module View……7
7. Security Design……8
8. Reporting & KPI Definitions……..9
9. API Design & Unified Error Shape……10
10. Database Design……..13
11. Third-party Integrations……..14
12. Frontend Information Architecture & Accessibility……..15
13. Testing Strategy & Quality Gates……..15
14. Deployment & Operations……..15
15. Activity Diagrams……..16

1. Executive Summary

TrinityDevApp which we called “Fresh Cart” is a mobile grocery platform built with a React Native (Expo) client, an Express REST API, PostgreSQL persistence, and a Nginx HTTPS gateway. The platform supports end-to-end customer flows (authentication, product discovery, barcode scanning, cart, checkout, and order history) and operational/admin flows (product management, order oversight, customer management, profile management, and favorites).

The runtime architecture is containerized via Docker Compose, and CI/CD pipelines validate tests and deploy backend artifacts. This document consolidates architecture, ADR decisions, API and data design, security controls, testing strategy, integrations, and deployment operations according to the project deliverable format.

2. Project Background & Objectives

The project addresses three core objectives:

1. **Customer experience**: deliver a clear mobile purchase journey from product search/scan to payment and order tracking.
2. **Operational control**: provide role-based administrative capabilities for managing products, users, and order visibility.
3. **Engineering quality**: maintain a secure, containerized, testable codebase with automated quality gates.

Success targets:

1. Secure JWT-based authentication and route protection are enforced on protected APIs.
2. Barcode workflows support lookup and scan-to-cart with explicit error codes.
3. Payments are processed through PayPal order creation/capture and then persisted as platform orders.
4. Global Jest coverage threshold remains at or above 40%.

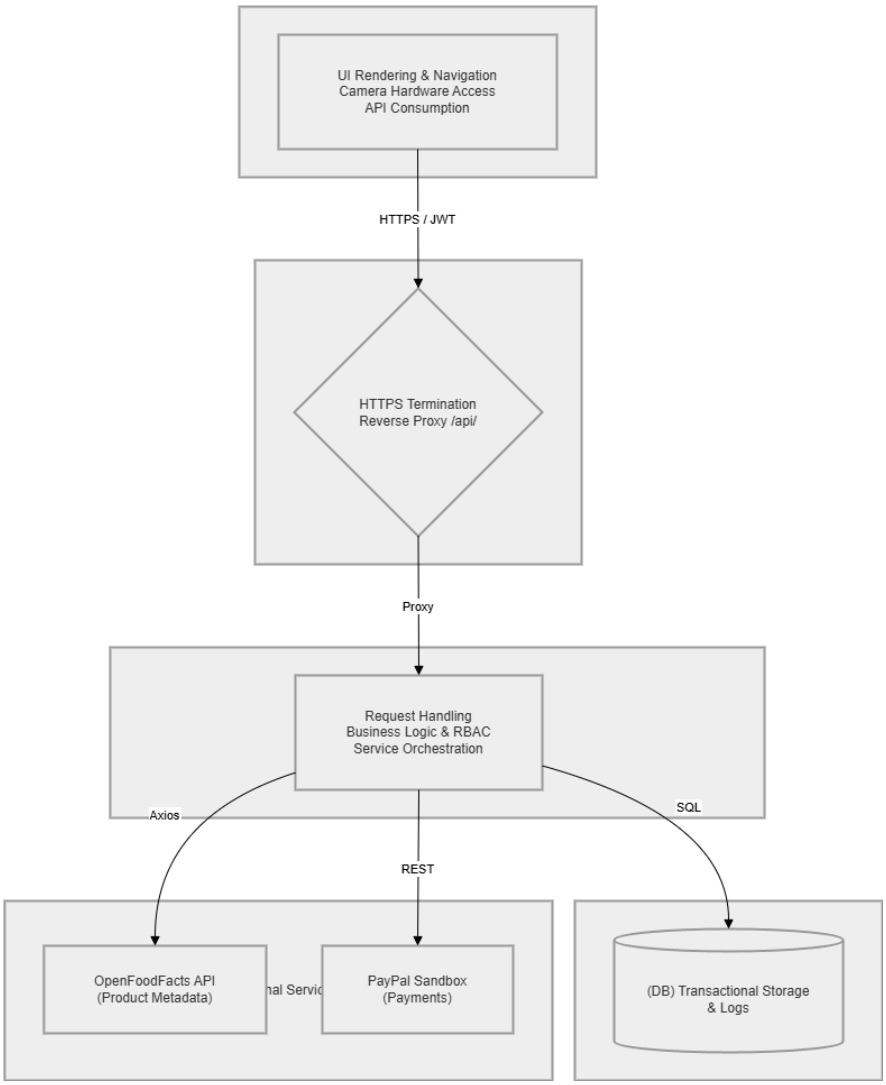
3. Requirements of Compliance Matrix

Requirement	Implementation in TrinityDevApp	Evidence
JWT Authentication	Secure login uses JWT bearer tokens for protected API access	'backend/controllers/auth.controller.js', 'backend/middleware/auth.js'
Refresh Session Flow	JWT session refresh/renewal is covered in the authentication lifecycle design	'frontend/lib/auth.ts', 'frontend/lib/api.ts'
Role-Based Access	Role guards enforce manager/admin and admin-only operations	'backend/middleware/roles.js', 'backend/routes/product.routes.js', 'backend/routes/order.routes.js'
Login + Signup	Email/password sign-in and account creation are fully available	'frontend/screens/LoginScreen.tsx', 'frontend/screens/RegisterScreen.tsx', 'backend/routes/auth.routes.js'
Home Overview + Navigation	Home dashboard and tab navigation provide fast access to scanner, cart, orders, and account	'frontend/screens/HomeScreen.tsx', 'frontend/App.tsx'
Promotions + Recommendations + Messages	Home includes promotional areas, recommendation blocks, and in-app user messaging via alert host	'frontend/screens/HomeScreen.tsx', 'frontend/lib/styledAlert.tsx'
Scanner Capability	Smartphone camera scanning with lookup and scan-to-cart is implemented	'frontend/screens/BarcodeScannerScreen.tsx', 'backend/routes/scanner.routes.js'
Scanner Error Handling	Unreadable barcode, invalid format, and product-not-found flows provide explicit feedback	'frontend/screens/BarcodeScannerScreen.tsx', 'backend/controllers/scanner.controller.js'
Product Display Detail	Product details include name, image, brand, category, nutrition, price, and stock	'frontend/screens/ProductDetailScreen.tsx', 'frontend/lib/api.ts'
Cart Management	Cart supports quantity changes, item removal, and live total calculations	'frontend/screens/CartScreen.tsx', 'backend/routes/cart.routes.js'
Billing Form + PayPal	Checkout form captures first name/last name/address/zip/city and completes payment via PayPal	'frontend/screens/CheckoutScreen.tsx', 'backend/routes/paypal.routes.js'
Purchase History	Previous purchases and transaction details are available in order history/detail screens	'frontend/screens/OrderHistoryScreen.tsx', 'frontend/screens/OrderDetailScreen.tsx', 'backend/routes/order.routes.js'

Camera Privacy + Performance	Runtime camera permissions, torch support, and real-time barcode decoding provide smooth UX	'frontend/screens/BarcodeScannerScreen.tsx'
Transaction Security	HTTPS transport + PayPal capture verification callback flow is implemented for payment integrity	'nginx/default.conf', 'backend/routes/paypal.routes.js', 'frontend/screens/CheckoutScreen.tsx'
Internal API + JWT Requests	Client API layer injects Authorization bearer token for authenticated requests	'frontend/lib/api.ts'
Caching/Redundant-call Reduction	AsyncStorage-backed cache/state and recommendation fallback reduce redundant API calls	'frontend/lib/api.ts', 'frontend/screens/HomeScreen.tsx'
UX/UI Practices	Friendly visual design, guided interactions, and animation-enhanced scanner experience are implemented	'frontend/screens/HomeScreen.tsx', 'frontend/screens/BarcodeScannerScreen.tsx'
Modular Architecture + Patterns	Modular route/controller/middleware + reusable frontend services/components with clear separation of concerns	'backend/routes/', 'backend/controllers/', 'frontend/lib/', 'frontend/screens/'
Architecture Documentation + Data Flow	Architecture layers and business workflows are documented with diagrams	See Section 4, Section 15
Unit Tests >= 60% + CI Visibility	Global 60% coverage gate and CI pipeline test jobs are configured and visible	'jest.config.js', '.github/workflows/CI-CD-pipeline.yml'
CI/CD Delivery	Dockerized test/build/deploy pipeline to EC2 is implemented	'.github/workflows/CI-CD-pipeline.yml'

4. Overall Architecture

The system follows a layered mobile-backend architecture:
1. **Mobile App (Expo)**: UI rendering, navigation, camera hardware access, and API consumption.
2. **Nginx Gateway**: HTTPS termination and reverse proxy to backend '/api/' routes.
3. **Express API**: request handling, business logic, auth/RBAC enforcement, and external service orchestration.
4. **PostgreSQL**: transactional storage for business entities and logs.
5. **External services**: OpenFoodFacts for product metadata and PayPal Sandbox for payment transactions.



System Architecture Overview

5. Technology Stack & Architectural Decisions (ADR)

1. Stack Overview

- Frontend: React Native 0.81, Expo 54, TypeScript, React Navigation, Expo Camera.

- Backend: Node.js runtime, Express 4.

- Operations: PostgreSQL 16, Nginx TLS gateway, Docker Compose, GitHub Actions.

- Container baseline: backend image currently uses Node 20 (node:20-alpine in Dockerfile).

2. Backend

- Expo + React Native for rapid cross-platform delivery and camera integration.
- Express for simple composable middleware and clear route/controller boundaries.

- PostgreSQL for relational constraints and reliable transactional writings.
- Nginx + Docker Compose for reproducible HTTPS deployment and service isolation.

6. Component & Module View

The system architecture follows a strictly modular design, ensuring a clear separation between the presentation layer (Mobile App), business orchestration (Express API), and the data persistence layer (PostgreSQL)

1. Frontend Module View (React Native / Expo)

The frontend is structured as a component-based mobile application utilizing **React Navigation** for workflow orchestration and the **Context API** for centralized state management.

Core Application Components

- Authentication & Identity: Handles user registration and secure login. It integrates form validation, password visibility toggles, and manages JWT persistence to ensure a seamless session across app restarts.
- Product Discovery & Search: Provides customers with high-performance browsing via category filters and keyword searches. It renders interactive product cards displaying pricing, stock levels, and nutritional grades.
- Intelligent Barcode Scanner: A specialized module utilizing expo-camera to decode barcodes in real-time. It includes an animated scanning UI and logic to fetch enriched data from the OpenFoodFacts API or the local database.
- Shopping Cart & Checkout: Manages transient state for selected items. It calculates totals in real-time and handles the complex PayPal integration flow, managing deep-link redirects and session state during payment.
- Admin Management Suite: An elevated-privilege domain for managers. It provides KPI visualization (Sales charts, User growth) and full CRUD capabilities for inventory management, allowing admins to update stock or adjust pricing instantly.
- Order & Profile History: A dedicated portal for users to track order statuses, view historical transaction details, and manage personal profile settings or "Favorites".

Shared Utility Modules:

- API Service Layer (Axios): A centralized client configured with interceptors to automatically attach JWT tokens to requests and handle global error responses.

- UI Component Library: A collection of reusable, styled components (Buttons, Inputs, Modals) ensuring visual consistency and accessibility (WCAG) across all screens.

2. Backend Module View (Express RESTful API)

The backend implements a Layered MVC Pattern, separating HTTP communication from core business rules to enhance testability and maintainability.

Layered Organization

- Routes Layer: Defines RESTful endpoints (Auth, Products, Cart, PayPal). It maps incoming URLs to specific controller functions and attaches Middleware chains for security.
- Middleware Module: Implements centralized security logic, including JWT identity verification and Role-Based Access Control (RBAC), ensuring that only admins can access sensitive inventory or customer data.
- Controllers Layer: Acts as the entry point for requests. It validates user input, orchestrates calls to service logic, and ensures all responses follow the Unified Error Shape.
- Business Services Layer: The "heavy lifting" logic of the system.
- Scanner & Sync Service: Handles logic for barcode lookups and ensures local product data is synchronized with external sources.
- PayPal Payment Proxy: Manages secure communication with PayPal's servers, handling order creation and transaction capture.
- Stock Management: Validates inventory availability during the checkout process to prevent over-selling.
- Models Layer (Data Objects): Defines the relational structures for Users, Products, Orders, and ScanEvents. It ensures data integrity through foreign key constraints and automated timestamps.

7. Security Design

1. Security Implementation Details

- Transport Layer Security (TLS): All communication between the mobile client and the AWS-hosted backend is encrypted using TLS 1.2/1.3 terminated at the Nginx gateway. Due to the absence of a registered domain, a Custom Self-Signed CA strategy was implemented to sign certificates for the AWS Elastic IP. The Android client is configured via a custom Expo plugin to trust this specific CA, ensuring end-to-end encryption even in a non-domain environment.

- Stateless Identity & Access Control: Authentication is handled via JWT (JSON Web Tokens). Upon login, the server issues a signed token which the client must include in the Authorization: Bearer <token> header for all protected API requests. Role-Based Access Control (RBAC) is strictly enforced via server-side middleware, distinguishing between Customer, Manager, and Admin roles to prevent unauthorized access to sensitive management endpoints.
- Input/Output Hygiene & Data Integrity: To prevent SQL Injection, all database interactions are performed using the ORM's parameterized queries. Centralized validation middleware (using express-validator or similar) sanitizes all incoming request bodies, while security-focused HTTP headers (via Helmet.js) are utilized to mitigate XSS and clickjacking attacks.
- Production Transport Enforcement: The backend includes a fail-safe check that monitors incoming traffic. If the HTTPS_REQUIRED flag is active and insecure traffic is detected without explicit override, the application rejects the request to prevent accidental data exposure in production environments.
- Secret & Infrastructure Security: Sensitive credentials, including the JWT_SECRET, PayPal Client ID/Secret, and database passwords, are managed via Environment Variables. For the CI/CD pipeline, these are stored in GitHub Secrets, ensuring that no credentials are ever hardcoded in the source code or visible in logs.

2. Threat and Mitigation Summary

Threat	Impact	Mitigation Strategy
Credential Sniffing	Compromise of user accounts	Enforced TLS 1.3 encryption and HSTS.
Broken Authorization	Customers accessing admin tools	Multi-level RBAC middleware on the backend.
SQL Injection	Data exfiltration or deletion	Mandatory use of ORM parameterized queries.
PayPal Session Hijacking	Fraudulent financial transactions	Backend-to-backend PayPal order capture with deep-link verification.
Brute Force Attacks	Account takeover	Implementation of rate-limiting headers and failed-login auditing.

8. Reporting & KPI Definitions

KPI	Definition	Window/Filter	Purpose
Average Order Value (AOV)	Total paid revenue / Total number of paid orders	Last 24h / 30 days	Measure average customer basket value.
Daily Orders Count	Total count of successful order transactions	Daily (24h)	Monitor real-time sales volume and server load.
Cart to Order Conversion	Users with paid orders / Users with cart activity	Last 14 days	Evaluate the efficiency of the checkout funnel.
Top Scanned Products	Products ranked by successful barcode scan counts	Last 7 days	Identify high-interest items via the mobile scanner.
Sell-through Rate	Total quantity sold / Initial inventory baseline	Monthly / Quarterly	Measure inventory efficiency and stock turnover.

9. API Design & Unified Error Shape

Endpoint Statistics:

- Total API route groups implemented: 9 (auth, products, cart, users, scanner, orders, openfoodfacts, paypal, favorites)
- Public endpoints: login/register, product list/detail, scanner lookup, openfoodfacts search/barcode, health test
- Protected endpoints: cart, order creation/history, profile/admin operations, favorites, scanner add-to-cart, PayPal order create/capture

1. Authentication & Sessions

Method	Path	Description	Auth
POST	/api/auth/register	Register account with email/password	Public
POST	/api/auth/login	Login and issue JWT token	Public

2. Products

Method	Path	Description	Auth
GET	/api/products	List product catalog	Public

GET	/api/products/:id	Get a single product	Public
GET	/api/products/discounted/list	List discounted products	Public
GET	/api/products/recommendations	Personalized recommendations	Required
POST	/api/products	Create product	Required (admin)
PUT	/api/products/:id	Update product	Required (admin)
PATCH	/api/products/:id/discount	Update product discount	Required (admin)
DELETE	/api/products/:id	Delete product	Required (admin)

3. Users

Method	Path	Description	Auth
GET	/api/users/profile	Get current user profile	Required
PUT	/api/users/profile	Update current user profile	Required
GET	/api/users	List users	Required (admin)
GET	/api/users/:id	Get user detail	Required (admin)
PUT	/api/users/:id/role	Update user role	Required (admin)
DELETE	/api/users/:id	Delete user	Required (admin)

4. Cart

Method	Path	Description	Auth
GET	/api/cart	Get current user cart	Required
POST	/api/cart/add	Add to cart	Required
PUT	/api/cart/update/:id	Update cart item	Required
DELETE	/api/cart/remove/:id	Remove cart item	Required
DELETE	/api/cart/clear	Clear cart	Required

5. Invoices/Orders

Method	Path	Description	Auth
POST	/api/orders	Create order from checkout	Required
GET	/api/orders/me	Get current user orders	Required

GET	/api/orders	Get all orders	Required (admin)

6. Scanner

Method	Path	Description	Auth
POST	/api/scanner/lookup	Lookup product by barcode	Public/Optional
POST	/api/scanner/add-to-cart	Scan and add product to cart	Required

7. PayPal

Method	Path	Description	Auth
POST	/api/paypal/create-order	Create PayPal checkout order	Required
POST	/api/paypal/capture-order	Capture approved PayPal order	Required

8. Favorites

Method	Path	Description	Auth
GET	/api/favorites	List user favorite products	Required
POST	/api/favorites	Add product to favorites	Required
DELETE	/api/favorites/:product_id	Remove favorite product	Required
GET	/api/favorites/check/:product_id	Check favorite status	Required

9. OpenFoodFacts Proxy

Method	Path	Description	Auth
GET	/api/openfoodfacts/search	Search OFF products by keyword/category	Public
GET	/api/openfoodfacts/barcode/:barcode	Fetch OFF product by barcode	Public

10. System

Method	Path	Description	Auth

GET	/api/test	Backend health/test endpoint	Public

11. Error Response Format

The API returns structured domain-specific JSON errors across modules (for example scanner returns code + message) and provides stable error handling for frontend consumption.

Scanner-style example:
{ "code": "PRODUCT_NOT_FOUND", "message": "No product found for this barcode."}

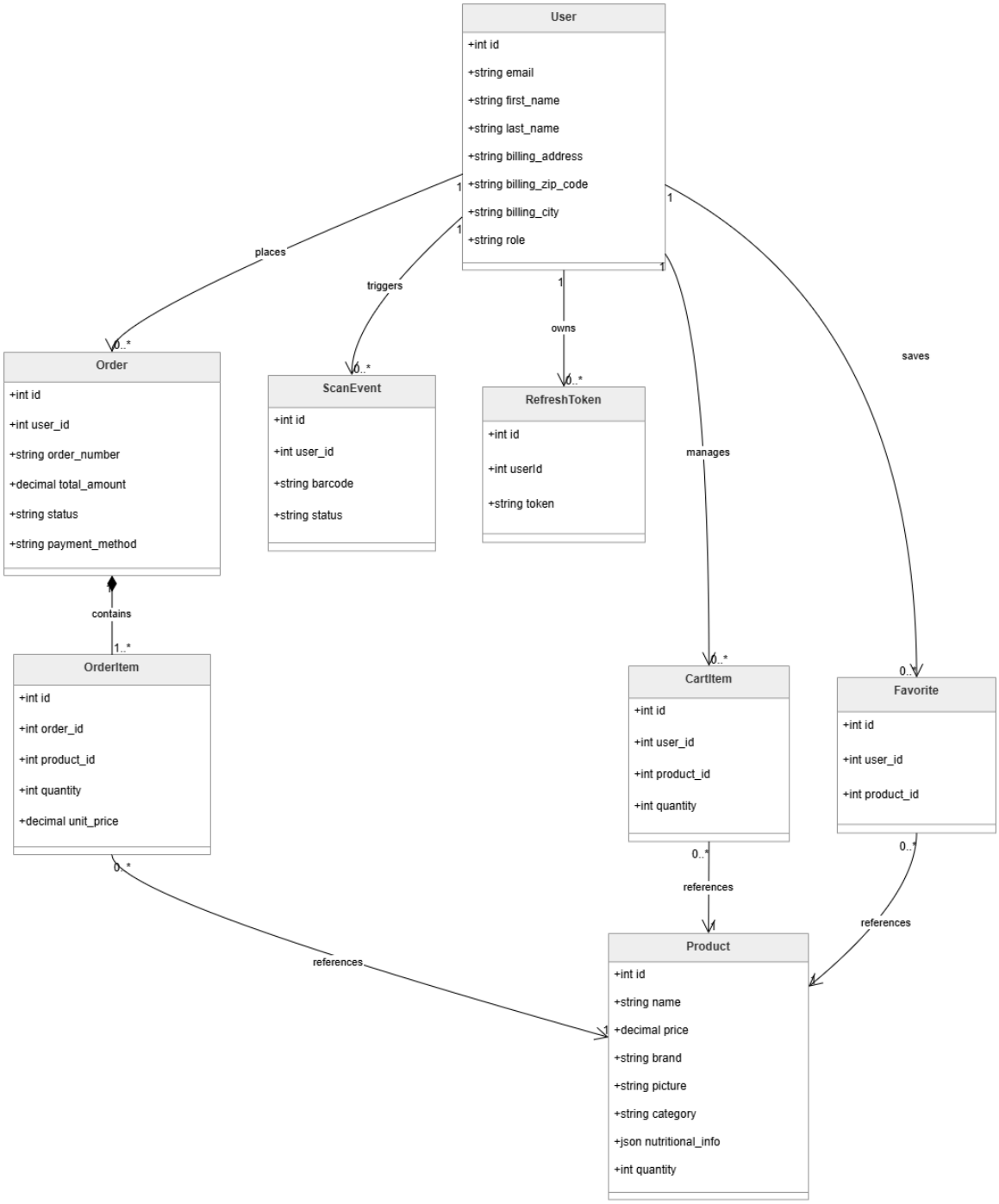
HTTP Status Code Mapping:

Status Code	Meaning	Typical Usage
200	OK	Query success / operation success
201	Created	Order or user creation success
400	Bad Request	Invalid payload/validation error
401	Unauthorized	Missing/invalid token
403	Forbidden	Role permission denied
404	Not Found	Product/order/resource not found
409	Conflict	Stock conflict or duplicated business condition
426	Upgrade Required	HTTPS required in production
500	Server Error	Internal processing failurer

10. Database Design

Core relational entities: users, products, cart, orders, order_items, invoices, invoice_items, favorites, scan_events.

Design characteristics:
1. Foreign keys with cascade/set-null policies maintain referential integrity.
2. Unique constraints protect users.email, users.username, orders.order_number, and favorite uniqueness per user/product.
3. Performance indexes exist for barcode, order lookup, and favorites lookup.



Class Diagram (Data Structure)

11. Third-party Integrations

1. OpenFoodFacts

- Backend proxy endpoints support search by text/category and lookup by barcode.
- Response content is transformed and encoding-repaired before returning to clients.

2. PayPal Sandbox

- Backend obtains PayPal access token via client credentials.
- Backend creates/captures PayPal orders.
- Frontend supports web and mobile payment flows:
  Web: @paypal/react-paypal-js
  Mobile: expo-web-browser + deep-link callback handling (freshcart:// or Expo URL)

12. Frontend Information Architecture & Accessibility

- Customer tabs Home, Search, Cart, Orders, Profile.
- Admin tabs Dashboard, Products, Orders, Customers, Profile.
- Stack routes scanner, category/product details, checkout, favorites, edit profile.
- Camera permission flow is explicitly handled before scan starts.
- Alert host centralizes user-facing feedback for errors and confirmations.
- Theme and accessibility preferences (including reduce motion usage in scanner animation) are integrated.

13. Testing Strategy & Quality Gates

- Frontend unit tests: TS/TSX tests under frontend/ with coverage collection on frontend/lib/.
- Backend unit tests: JS tests under backend/ with coverage collection on controllers and middleware.
-Coverage threshold: global minimum 20% for branches, functions, lines, and statements.
-CI execution: GitHub Actions runs backend then frontend coverage in Dockerized test containers.
User-facing UX validation is continuously performed during feature testing (login, scanning, cart updates, checkout, and order history) to verify usability and interaction smoothness.

14. Deployment & Operations

1. Runtime Deployment Model

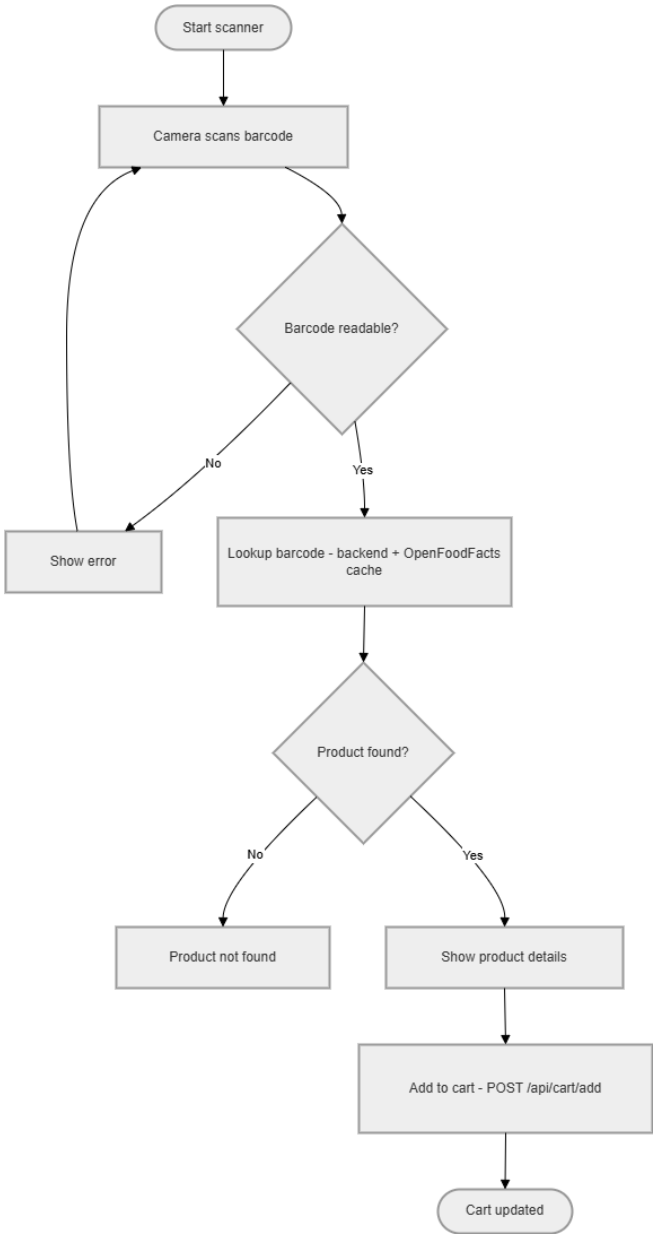
- db initializes schema from database/schema.sql.
- backend runs on internal network and is exposed via nginx reverse proxy.
- nginx serves HTTPS (443), redirects HTTP (80), proxies /api/ to backend.

2. CI/CD and Infrastructure

- CI validates backend/frontend tests.
- Backend image is built and pushed to Docker Hub.
- Production deployment job ships compose/schema/env files and performs remote docker compose up over SSH to EC2.
- App API defaults include a deployed HTTPS endpoint (https://13.37.46.130/api) for production builds.

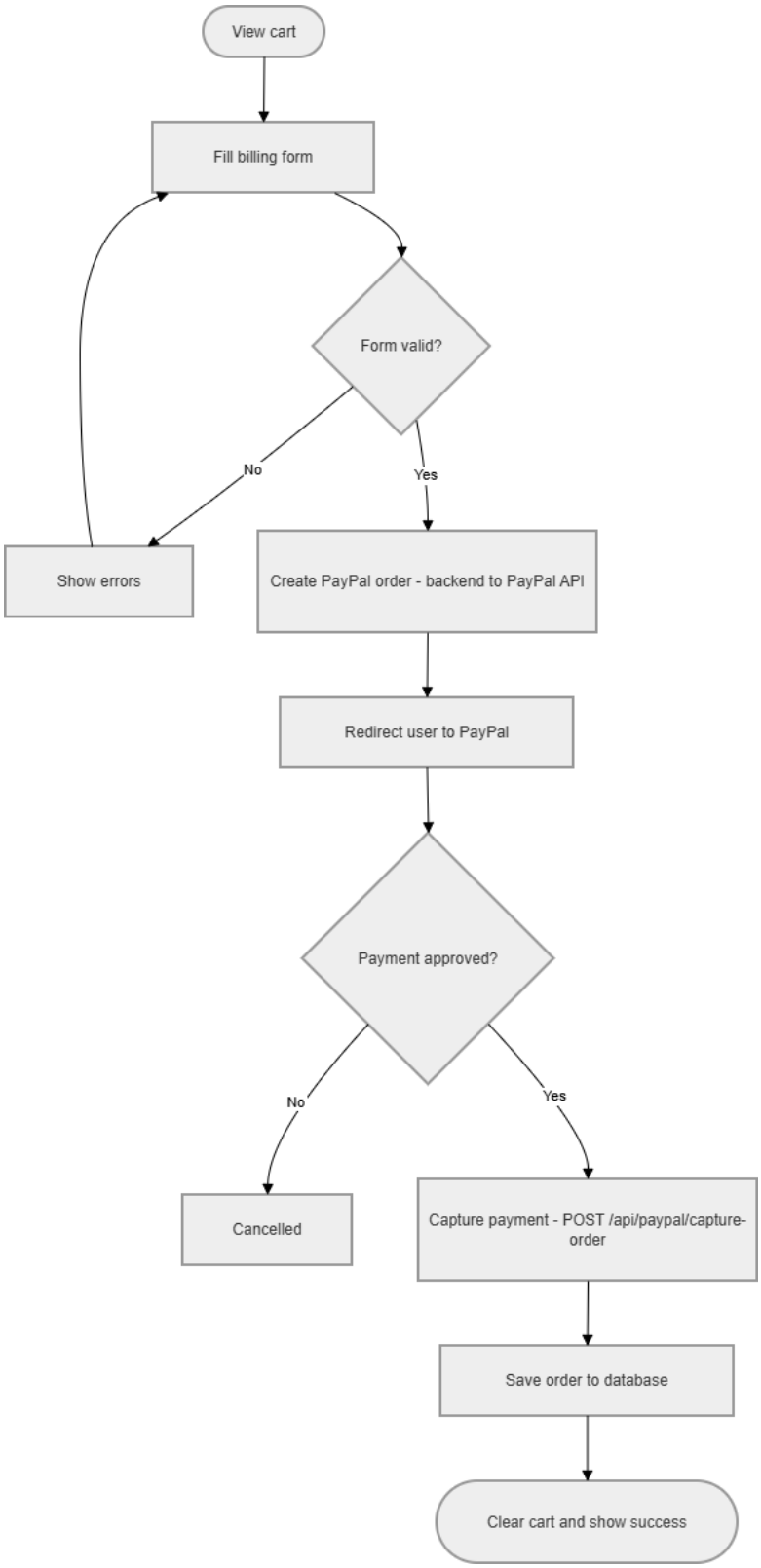
15. Activity Diagrams

1. Scan-to-Cart Flow



Scan-to-Cart Activity Flow

2. Checkout and Payment Flow



Checkout and Payment Activity Flow